Java + Spring Boot Interview Pack

Target level: Junior to Mid-level backend engineer, around 1 year of experience.

Purpose: quick revision pack for interviews, practical explanation of workflow, and internal Spring Boot understanding.

Aligned to the final scope of Spring Boot Learning Phase 01 and focused on Days 1-17 topics.

JAVA CORE | SPRING BOOT | JPA | SECURITY | TESTING | WORKFLOW

1. How To Present Yourself

Short intro sample: I am a Java backend developer with around 1 year of experience working with Spring Boot, REST APIs, JPA, validation, security, and testing. I have built CRUD APIs with layered architecture, JWT-based authentication, role-based access control, pagination, exception handling, logging, Swagger/OpenAPI, profile-based configuration, and multiple test layers including unit, slice, integration, and coverage reporting.

What you should emphasize

- Layered architecture: controller, service, repository
- DTO usage instead of exposing entities directly
- Validation and proper HTTP status codes
- Security basics: JWT, roles, 401 vs 403
- Testing mindset, not only feature building

What you should avoid

- Claiming deep microservices or cloud expertise if you have not used them
- Only explaining annotations without explaining request flow
- Talking only about happy-path CRUD
- Ignoring tradeoffs and error handling

2. Real Request Workflow In Spring Boot

Typical flow: Client sends HTTP request -> Spring Security filter chain checks authentication -> DispatcherServlet routes request -> controller receives DTO -> validation runs -> service executes business logic -> repository talks to database -> service returns response DTO -> controller sends HTTP response.

Layer	Main responsibility	Interview explanation
Controller	Handle HTTP request/response	Should stay thin. Accept request DTOs, call service, return response entity or DTO.
Service	Business logic	Best place for rules, orchestration, transactions, validations beyond field-level constraints.
Repository	Data access	Only persistence concerns. No request handling logic here.
Database	Store data	JPA/Hibernate maps Java entities to relational tables.

Good interview line: I try to keep controllers thin, put business decisions in services, and keep repositories focused on persistence. That makes code easier to test and maintain.

3. Spring Boot Internals You Should Know

Q: What does Spring Boot do for you?

A: Spring Boot reduces configuration overhead. It uses auto-configuration, starter dependencies, embedded server support, externalized configuration, and convention over configuration so we can focus on business logic faster.

Q: What is auto-configuration?

A: Auto-configuration means Spring Boot creates beans automatically based on the dependencies and properties available in the classpath. For example, if Spring Web is present, it configures MVC components; if JPA and a datasource are present, it configures JPA infrastructure.

Q: What is the role of DispatcherServlet?

A: DispatcherServlet is the front controller in Spring MVC. It receives incoming HTTP requests, finds the correct controller method through handler mapping, invokes it, and builds the response.

Q: What is dependency injection?

A: Dependency injection means objects receive their dependencies from the Spring container instead of creating them manually. It improves testability and loose coupling. Constructor injection is preferred because dependencies are explicit and immutable.

Q: What happens when the application starts?

A: Spring Boot creates the application context, scans components, registers beans, applies auto-configuration, starts the embedded Tomcat server, and initializes supporting infrastructure like JPA, Flyway, and security filters.

4. Java Core Questions With Good Answers

Q: Difference between JDK, JRE, and JVM?

A: JVM runs Java bytecode. JRE includes JVM plus runtime libraries. JDK includes JRE plus development tools like javac and javadoc.

Q: Why is Java platform independent?

A: Java source code is compiled into bytecode, and that bytecode runs on any platform that has a JVM.

Q: Difference between HashMap and ConcurrentHashMap?

A: HashMap is not thread-safe. ConcurrentHashMap supports concurrent access and is safer in multithreaded scenarios. HashMap allows one null key, ConcurrentHashMap does not allow null keys or values.

Q: Why use Optional?

A: Optional makes absence of value explicit and helps reduce accidental null handling mistakes. In Spring Data JPA, repository methods like findById often return Optional to represent possible absence of data.

Q: Difference between checked and unchecked exceptions?

A: Checked exceptions must be handled or declared at compile time. Unchecked exceptions extend RuntimeException and usually represent programming or domain errors that are not forced by the compiler.

5. JPA and Database Questions

Q: What is JPA and what is Hibernate?

A: JPA is a Java specification for ORM. Hibernate is a popular implementation of JPA. We code mostly against JPA APIs and Hibernate executes the ORM behavior underneath.

Q: What is the difference between save() and saveAndFlush()?

A: save() persists changes in the persistence context and the flush may happen later. saveAndFlush() forces synchronization to the database immediately. save() is enough in most cases.

Q: Why should you avoid ddl-auto=update in serious projects?

A: Because schema changes become implicit and hard to track. It is safer to use Flyway or Liquibase so schema evolution is explicit, versioned, and repeatable across environments.

Q: What is Flyway?

A: Flyway is a database migration tool. It runs versioned SQL scripts like V1__create_users_table.sql during startup so the database schema is managed in source control.

Q: What is the N+1 problem?

A: It happens when one query loads a list of parent entities and then additional queries are executed for each child relation. It causes performance issues and is commonly solved with better fetching strategies or join fetch queries.

---

6. Security Questions

Q: What is JWT?

A: JWT is a signed token containing claims such as subject and role. The server verifies the token signature and extracts claims instead of storing session state on the server.

Q: What is the difference between authentication and authorization?

A: Authentication answers who the user is. Authorization answers what the user is allowed to do.

Q: Difference between 401 and 403?

A: 401 means the request is not authenticated. 403 means the user is authenticated but does not have permission for that resource.

Q: Why do we hash passwords?

A: Passwords should never be stored in plain text. Hashing with BCrypt protects user credentials if the database is leaked and supports secure password verification.

Q: What does a JWT filter do?

A: It intercepts incoming requests, reads the Authorization header, validates the token, extracts user details or roles, and sets authentication into the SecurityContext so Spring Security can authorize the request.

7. Testing Questions

Q: Difference between unit test, slice test, and integration test?

A: Unit test isolates one class and mocks dependencies. Slice test loads only a focused framework layer such as MVC or JPA. Integration test loads more of the real application and checks components working together.

Q: What is @WebMvcTest used for?

A: It loads controller-related MVC infrastructure only. It is useful for fast controller testing with mocked services.

Q: What is @DataJpaTest used for?

A: It loads repository and JPA-related components only. It is used to test database queries and entity mapping behavior.

Q: Why use MockMvc?

A: MockMvc lets us test HTTP request and response behavior without deploying the application to a real external server. It is useful for verifying status codes, JSON, validation, and security rules.

Q: What does code coverage tell you and what are its limits?

A: Coverage tells us how much code was executed by tests, for example through a JaCoCo report. It is useful for spotting untested areas, but high coverage alone does not prove good assertions or real business-case testing.

8. Common Project-Based Questions

Q: Why did you use DTOs?

A: DTOs decouple the API contract from the entity model. They help control exposed fields, support validation, and avoid returning sensitive fields like passwords.

Q: Why did you add a global exception handler?

A: To keep error responses consistent and avoid repeating try/catch logic in controllers. It also helps return proper status codes like 400, 404, 409, 401, and 403.

Q: Why did you add pagination?

A: Returning the full dataset does not scale. Pagination reduces memory use, response size, and DB/network overhead, and gives clients controlled access to large datasets.

Q: How do you approach adding a new API in Spring Boot?

A: First define the API contract and DTOs. Then add controller endpoint, implement business logic in service, add repository logic if needed, define validations and error cases, secure the endpoint, and finally write unit, slice, and integration tests.

Q: Why did you use profiles like dev and prod?

A: Profiles let us keep environment-specific settings separate. For example, dev can use H2 and verbose logging, while prod can use PostgreSQL, safer defaults, and no local-only tools like the H2 console or seeded demo users.

Q: Why did you add Swagger / OpenAPI?

A: Swagger improves API discoverability and speeds up testing and collaboration. It gives a live contract for endpoints, request bodies, and response structures, which is useful both during development and in interviews when explaining the project.

Q: What do you log in a backend application?

> A: I log meaningful application events such as login attempts, entity creation, important failures, and request flow checkpoints. I avoid logging secrets, passwords, or raw tokens, and I use logs mainly for debugging and operational visibility.

---

9. Workflow You Should Explain In Interviews

Feature workflow

- Understand business requirement
- Identify request/response contract
- Design DTOs and validation rules
- Implement controller and service logic
- Use repository for persistence
- Add exception handling and security rules
- Write tests and verify with API calls

Bug fix workflow

- Reproduce the issue clearly
- Check logs and stack trace
- Narrow it to controller, service, repository, or config
- Fix root cause, not only symptom
- Add or update tests to prevent regression

Code review workflow

- Check correctness first
- Check API contract changes
- Check security and validation
- Check DB impact and migration need
- Check test coverage and edge cases

---

10. Internal Debugging Mindset

Problem	How to think	Likely area
400 Bad Request	Was validation triggered? Is JSON shape correct?	DTO, validation, request body

Problem	How to think	Likely area
401 Unauthorized	Was authentication missing or token invalid?	Security config, JWT filter
403 Forbidden	Was the user authenticated but missing a role?	Method security, authorities
404 Not Found	Does data exist? Is endpoint mapping correct?	Controller path, service lookup
500 Internal Server Error	Check stack trace, logs, null handling, configuration	Service logic, DB, config, unexpected exceptions

11. Strong Answers About Your Current Repo

If they ask you about a real project, use your current Spring Boot repository.

> **Q: What did you build in your project?**
>
> A: I built a Spring Boot backend with user management, authentication, JWT security, role-based authorization, validation, exception handling, pagination, search, Flyway migrations, and multiple test layers.

> **Q: What improvements did you make over time?**
>
> A: I started with basic CRUD, then introduced service and repository layers, DTOs, validation, JPA persistence, security, structured error handling, logging, Swagger, Flyway, and full test coverage including unit, repository, controller slice, and integration tests.

> **Q: What did you learn from testing?**
>
> A: I learned to separate testing levels. Unit tests verify service rules, @WebMvcTest verifies controller behavior, @DataJpaTest verifies query behavior, and integration tests verify the whole request flow including security and persistence.

12. Day 1-17 Interview Drill

Use this section for focused revision if the interviewer asks you to explain your learning journey step by step.

Day	Main topic	Question they may ask	What your answer should emphasize
Day 1	Spring Boot setup	How do you start a Spring Boot project and what does `@SpringBootApplication` give you?	Mention auto-configuration, component scanning, starter dependencies, and embedded server support.
Day 2	First REST endpoint	What is the difference between `@Controller` and `@RestController`?	Explain JSON responses, HTTP endpoints, and that `@RestController` is effectively `@Controller` plus `@ResponseBody`.
Day 3	CRUD basics	How would you build a simple CRUD API in Spring Boot?	Walk through endpoint mapping, request DTO, service call, repository call, and proper status codes.
Day 4	Layered architecture	Why should controllers stay thin?	Say business logic belongs in services, not controllers, because it improves reuse, readability, and testing.
Day 5	JPA repository	What does `JpaRepository` give you out of the box?	Mention save, findById, findAll, deleteById, paging support, and custom derived queries.
Day 6	DTO design	Why not expose your entity directly in API responses?	Talk about API contract control, hiding sensitive fields like passwords, and separating persistence from API shape.
Day 7	Validation	How does validation work for incoming requests?	Explain Bean Validation annotations, `@Valid`, and how invalid payloads return 400 with structured errors.
Day 8	Exception handling	Why use a global exception handler?	Highlight consistent error JSON, centralized handling, and cleaner controllers with correct status codes.

Day	Main topic	Question they may ask	What your answer should emphasize
Day 9	Database config	Why use H2 in development and PostgreSQL-style config for production?	Explain fast local setup for learning and realistic production-ready database separation through profiles.
Day 10	Flyway migrations	Why are versioned SQL migrations better than relying on automatic schema updates?	Mention traceability, team consistency, repeatability across environments, and safer deployment workflow.
Day 11	Password security	How do you store and verify passwords securely?	Say passwords are hashed with BCrypt, never stored in plain text, and checked with a password encoder.
Day 12	JWT authentication	What happens after a user logs in?	Explain credential validation, JWT generation, token claims like subject and role, and stateless authentication.
Day 13	RBAC	How do you restrict certain APIs to admins only?	Mention authorities, roles, Spring Security context, and method-level checks such as `@PreAuthorize`.
Day 14	Pagination and search	Why use `Pageable` instead of returning everything?	Talk about scalability, response size, sorting, and better client-side control when datasets grow.
Day 15	Logging and Swagger	How do logs and Swagger help during development?	Logs help debugging and operational tracing; Swagger helps discover, test, and explain APIs quickly.
Day 16	Profiles and environments	What changes between dev and prod profiles in your project?	Mention datasource settings, logging level, H2 console availability, seed data, and safer production defaults.
Day 17	Testing and coverage	What test layers do you have, and how do you measure coverage?	Explain unit, repository, WebMvc, integration, and security-focused tests, plus

Day	Main topic	Question they may ask	What your answer should emphasize
			JaCoCo reports for visibility into untested code paths.

Day 1-17 Detailed Questions and Answers

Q: Day 1 - How do you start a Spring Boot project and what does `@SpringBootApplication` give you?

A: I usually start from Spring Initializr and choose the dependencies I need such as Spring Web, JPA, Validation, Security, and H2 or PostgreSQL. `@SpringBootApplication` is a convenience annotation that combines configuration, component scanning, and auto-configuration. It helps Spring Boot find beans, configure the application based on classpath dependencies, and start the embedded server quickly.

Q: Day 2 - What is the difference between `@Controller` and `@RestController`?

A: `@Controller` is generally used for MVC applications that return views, while `@RestController` is used for REST APIs and returns data directly as JSON. In simple terms, `@RestController` is `@Controller` plus `@ResponseBody`. Since my project is a backend API project, `@RestController` is the correct choice.

Q: Day 3 - How would you explain a basic CRUD API in Spring Boot?

A: CRUD means create, read, update, and delete. In Spring Boot, I define endpoints in the controller, call service methods for business logic, and use a repository for database access. For example, `POST /users` creates a user, `GET /users/{id}` reads one user, `PUT /users/{id}` updates it, and `DELETE /users/{id}` removes it. I also return proper HTTP status codes like 201 for create, 200 for reads and updates, and 204 for delete.

Q: Day 4 - Why should controllers stay thin?

A: Controllers should mainly handle HTTP concerns such as request mapping, input binding, validation triggering, and response creation. If business logic is placed inside controllers, code becomes harder to test, reuse, and maintain. I keep decisions

such as duplicate checks, search logic, and update rules in the service layer so the architecture stays clean.

Q: Day 5 - What does `JpaRepository` give you out of the box?

A: `JpaRepository` provides common database operations such as `save`, `findById`, `findAll`, `deleteById`, paging, sorting, and count methods. It reduces boilerplate because I do not need to write basic CRUD SQL manually. I can also add custom derived queries like `findByEmail` or `findByNameContainingIgnoreCase`.

Q: Day 6 - Why did you introduce DTOs instead of exposing entities directly?

A: DTOs let me control the API contract independently from the database entity. They help me avoid exposing sensitive fields such as passwords, and they are the right place for request validation rules. They also make it easier to change the internal data model later without breaking clients.

Q: Day 7 - How does validation work in your project?

A: I use Bean Validation annotations such as `@NotBlank`, `@Email`, and `@Size` inside request DTOs. In controller methods I use `@Valid` so Spring validates the request body automatically before business logic runs. If validation fails, the request does not continue to the service layer and the API returns a 400 Bad Request with a structured error message.

Q: Day 8 - Why use a global exception handler?

A: A global exception handler keeps error handling centralized and consistent. Instead of repeating try/catch in every controller, I map exceptions like not found, duplicate resource, validation failure, unauthorized access, and forbidden access into one standard JSON structure. This improves API consistency and makes controllers cleaner.

Q: Day 9 - Why use H2 for development and PostgreSQL-style settings for production?

A: H2 is useful in development because it is fast and easy to run with no external setup. It helps when learning or testing features quickly. PostgreSQL-style configuration is closer to real production behavior, so using profile-based config lets me keep development simple while still preparing the project for more realistic deployment environments.

Q: Day 10 - Why is Flyway better than relying on automatic schema updates?

A: Flyway makes schema changes explicit, versioned, and repeatable. Instead of depending on Hibernate to change the database automatically, I keep migration scripts in source control. That means every environment can move through the same schema history safely, and the team can review database changes just like code changes.

Q: Day 11 - How do you store passwords securely?

A: I never store raw passwords in the database. I hash them using BCrypt through Spring Security's `PasswordEncoder`. During login, the raw password from the request is compared with the stored hash, and if they match, authentication succeeds. This protects user credentials even if the database is exposed.

Q: Day 12 - What happens after a user logs in?

A: The login endpoint accepts a request with email and password. The service looks up the user, checks the password hash, and if the credentials are valid, it generates a JWT containing claims such as the user email and role. The client then sends that token in the Authorization header on later requests, and the server validates it instead of using session state.

Q: Day 13 - How do you implement role-based access control?

A: I store the user's role, include it in the JWT, and extract it inside the JWT authentication filter. After that, Spring Security places the authenticated user and authorities into the security context. Then I use method-level rules such as `@PreAuthorize("hasRole('ADMIN')")` to protect admin-only endpoints. That is how the API distinguishes authenticated users from authorized users.

Q: Day 14 - Why did you add pagination and search?

A: Returning every record does not scale well as data grows. Pagination keeps response sizes controlled and reduces memory and database load. Search endpoints improve usability by letting clients filter users by fields like name or email. In Spring Data, `Pageable` makes this easy because it combines page number, page size, and sorting.

Q: Day 15 - Why are logging and Swagger useful?

A: Logging helps me debug application behavior and follow important events such as login attempts, user creation, and unexpected failures. Swagger or OpenAPI helps document the API, test endpoints quickly, and explain the project clearly in interviews. Together they improve both developer productivity and maintainability.

Q: Day 16 - What changes between dev and prod profiles in your project?

A: In the dev profile I use local-friendly settings like H2, in-memory caching, H2 console access, and seed data for quick testing. In the prod profile I use PostgreSQL-style configuration, safer logging defaults, and I avoid development-only tools and seeded demo data. Profiles help keep one codebase but separate environment behavior cleanly.

Q: Day 17 - What kinds of tests do you have and how do you measure coverage?

A: I use unit tests for service logic, repository tests for JPA behavior, `@WebMvcTest` for controller slice testing, and `@SpringBootTest` integration tests for end-to-end request flow. I also added security-focused tests for unauthorized, forbidden, and invalid-token scenarios. For coverage, I use JaCoCo to generate a report so I can see which classes and lines are exercised and where gaps remain.

Interview strategy: If they ask what changed over 17 days, answer in layers: API basics -> architecture -> persistence -> validation -> security -> observability -> testing.

Phase 01 Completion Summary

> **Q: What did you finish in Spring Boot Phase 01?**
>
> A: I completed the core backend learning path: Spring Boot setup, REST APIs, layered architecture, DTOs, validation, exception handling, JPA, Flyway, JWT authentication, role-based authorization, pagination, search, logging, Swagger, profile-based configuration, and multiple testing layers with coverage reporting. I intentionally stopped there because that already covers the fundamentals I need to explain confidently in interviews.

> **Q: Why did you stop the project at this point?**
>
> A: Because the goal was learning Spring Boot fundamentals, not building an endless feature list. At this point the repository already demonstrates the main concepts a junior backend developer should understand clearly. Adding unrelated features would increase size, but not necessarily improve my understanding of the core stack.

---

13. Rapid Fire Questions

- What is @RestController? Combination of @Controller and @ResponseBody.
- What is @Service? Marks a business-layer component managed by Spring.
- What is @Repository? Persistence layer component; can translate persistence exceptions.
- What is ResponseEntity? Gives control over body, headers, and status code.
- What is @Valid? Triggers Bean Validation on request objects.
- What is Pageable? Spring Data abstraction for pagination and sorting.
- What is @PreAuthorize? Method-level authorization based on expressions and roles.
- What is application.properties? Externalized configuration file for Spring Boot.
- What is a bean? An object managed by the Spring container.
- What is IoC? Inversion of Control; the container manages object creation and wiring.

---

14. Questions You Can Ask the Interviewer

- How is the backend team structured?
- How do you handle code reviews and testing?
- Do you use monoliths, microservices, or a mixed approach?
- How are deployments handled across environments?

- What is expected from a developer in the first 3 months?

15. Final Revision Checklist

- Be able to explain one full request flow clearly from controller to database
- Know why DTOs, services, and repositories are separated
- Know 400 vs 401 vs 403 vs 404 vs 409 vs 500
- Know JWT basics and password hashing
- Know JPA basics and why Flyway is better than ddl-auto update
- Know test types and where each one fits
- Be able to explain your own project confidently

Prepared for interview revision and aligned to the completed scope of Spring Boot Learning Phase 01. Best used as a speaking guide, not as a script to memorize word-for-word.